Statistics1Key

September 24, 2019

Welcome to Python!

Python is a very powerful yet easy to use programming language that has become one of the most commonly used languages, especially in science. There are a variety of ways to use Python. You can run Python in a command line structure and type out commands. You can also write scripts and then run the whole script at once.

Here we are going to use a Python notebook run in the cloud. This is sometimes called iPython or Jupyter.

Some cells, like this, are text only (in a format called Markdown). I'll use these to type out things to know.

Other cells are executable blocks of code. You can run them in different orders and rerun bits of code, but the results of the output will liven in the memory, as you will see later.

First, replace the ... in the print command below with your name, and hit Shift + Enter.

One important note: this needs to be Python 3. If you see Python 2 in the upper right corner, switch it using Kernel > Change Kernel.

```
In [2]: print("Hello World! My name is Michael")
```

```
Hello World! My name is Michael
```

Let's start learning how to program. This block of code shows you how to define variables. Once you store these, you can recall them later.

Run this block of code by clicking run or Shift + Enter or Ctrl + Enter.

```
In [3]: a=1
        b=2
        c=a+b
        print("c = ", c)
        # Hit shift + enter to see the result: c = 3
```

```
c =  3
```

Here, a is stored as the number 1, and b is stored as the number 2. We can save c as the sum of a and b. The print() command prints various things. Using quotes tells program to consider what is inside as text, which we call a string.

Using a # sign at the beginning of the line tells the program that this is just a comment, not actual code, and it will ignore it.

As the comment says, hitting Shift + Enter will run the block of code.

Other common mathematical operations are performed using -, *, and /.

Problem 1: Use the next code block to define d as the product of a and b and print the result. Note, you don't need to define a and b again.

```
In [19]: # Your work here...

         d=a*b
         print("d=", d)
```

```
d= 2
```

Python is good at calculating single variables, but the main advantage is that is can work with a lot of data at once, which is difficult to do by calculator or even Excel.

A list in Python is simply a collection of things, which can be variables, numbers, or string.

An array is like a list but is more useful because operations can be performed on the whole array.

To use array, we need to import a library called Numerical Python or NumPy. After you import it by running this block of code, you don't need to run it again.

This example will show you how to build an array and use it to do mathematical operations.

```
In [20]: import numpy as np

         x = np.array([1,2,3,4,5,6])
         y = x ** 2
         print(y)
```

```
[ 1  4  9 16 25 36]
```

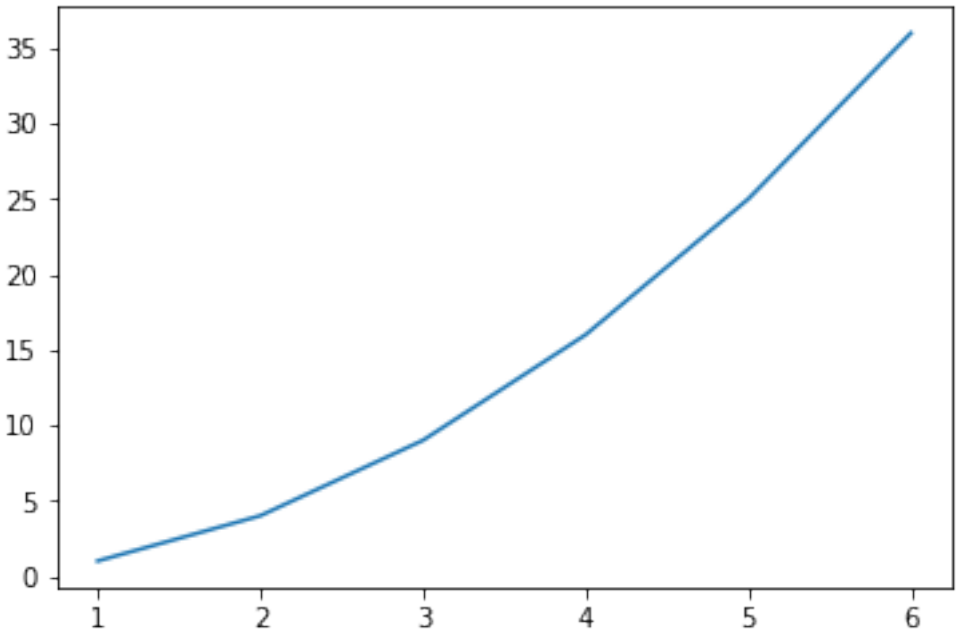
You can see we built an array using the command np.array() with 1 through 6 in it. We used the ** operation to square the values in the array.

We have imported numpy as np, so we can use numpy functions by calling np.. We will learn a handful of numpy functions throughout this tutorial.

Now, let's learn how to plot this data. For this, we need to import another library, MatPlotLib, and we are going to use the pyplot submodule of it. We will rename this as plt to make it easy to access.

```
In [21]: import matplotlib.pyplot as plt

         plt.plot(x,y)
         plt.show()
```



Note, if this plot doesn't work, make sure you ran the input above to define x and y. Otherwise, it won't know what to plot.

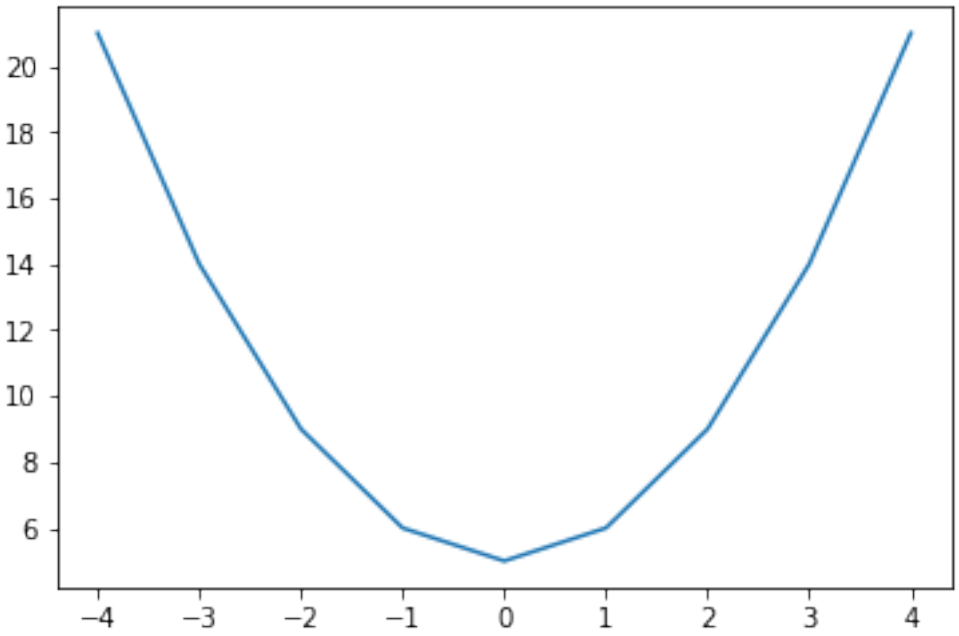
You can see that we generated a plot of x versus y, which should have a classic parabola form. We need to include the show() command to make it print out the plot.

Plotting the results is a really useful way to see what your data looks like and debug things.

Problem 2: Create an array to plot x**2 + 5 from -4 to 4.

```
In [22]: # Your code here
         x = np.array([-4,-3,-2,-1,0,1,2,3,4])
         y = x**2 + 5

         plt.plot(x,y)
         plt.show()
```



Often, you want to write functions to simplify things. This is done using def, defining the function name and (inputs), and using return to specify the outputs.

Note you need a colon after the function definition and everything inside the function needs to be tabbed. The first untabbed line will exit the function.

This function simply defines add and returns the sum of two numbers. You call it using add() and inserting the two numbers.

```
In [23]: def add(a, b):
             c = a+b
             return c

         print(add(4,5))
```

```
9
```

This function defines a Gaussian Distribution (equation 4.3 in the textbook). It makes use of a few new numpy functions: np.sqrt (Square Root), np.exp (Exponential or e^x), and np.pi (simply the constant Pi).

We then define x using another numpy function, np.arange, which creates an array of values ranging from the minimum to the maximum in increments of 1, which is the default.

If any of these functions are confusing, add in a print command to see what the outputs look like.

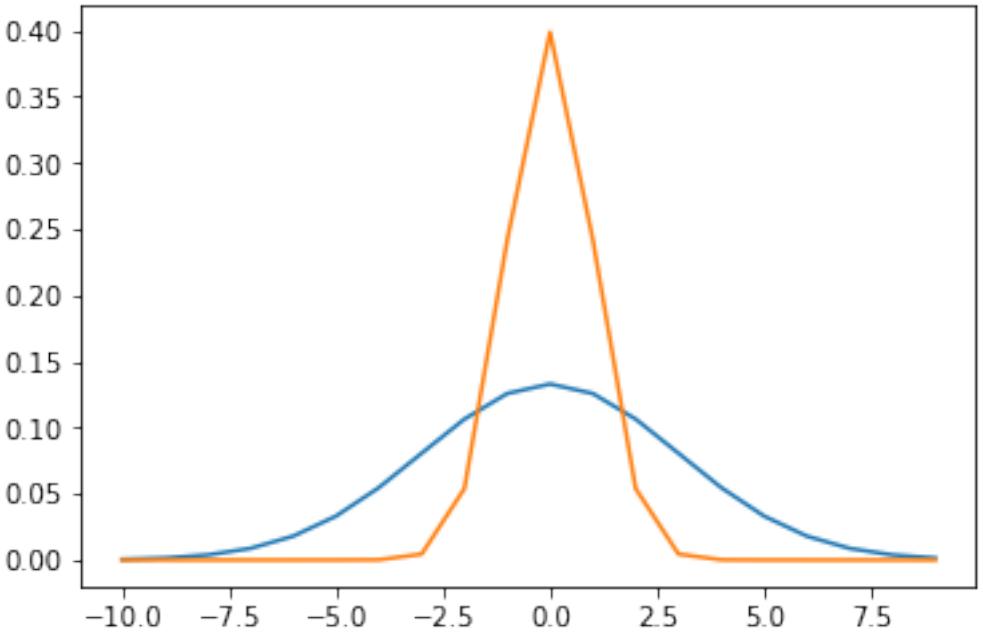
We can define y and z as the outputs of the Gaussian function with different inputs.

```
In [24]: def Gaussian(xvalues, mean, standard_deviation):
             sdsquared = standard_deviation**2
```

```
    exponent = -(xvalues - mean)**2/(2 * sdsquared)
    preexponential = 1 / np.sqrt(2 * np.pi * sdsquared)
    return preexponential * np.exp(exponent)

x = np.arange(-10, 10)
y = Gaussian(x, 0, 3)
z = Gaussian(x, 0, 1)

plt.plot(x, y)
plt.plot(x, z)
plt.show()
```



Gaussian Distributions are defined by two parameters: the mean and the standard deviation. The mean specifies where the center of the distribution is. The standard deviation specifies the width of the peak.

The above block of code also shows you how to plot multiple things on the same graph by just adding extra plot commands before you call show().

This plot shows an example of two Gaussian distributions with the same mean but different standard deviations. Notice, the area under both curves should be 1, which is why the peak intensities are different.

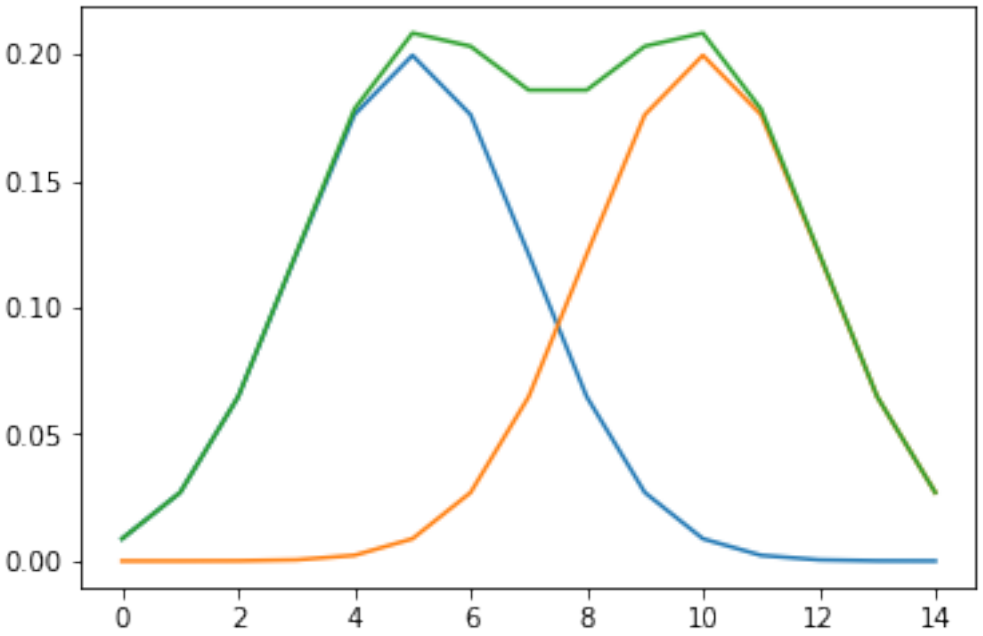
Problem 3: Plot two Gaussian distributions with a standard deviation of 2 and means of either 5 or 10. Challenge problem: Also plot the sum of the two distributions, which is done by simply adding the arrays (y+z). Note: you will want to extend the x values to hit the whole range.

```
In [25]: # Your work here...

         x = np.arange(0, 15)
```

```
y = Gaussian(x, 5, 2)
z = Gaussian(x, 10, 2)
sum = y+z

plt.plot(x,y)
plt.plot(x,z)
plt.plot(x,sum)
plt.show()
```



A lot of analytical data take the form of a Gaussian, so it is important that we can calculate these parameters.

Mean = Sum of X / number of data points in X

Let's calculate the mean using a few programmatic tricks. Here, np.sum() allows us to quickly sum the array and len() lets us calculate how long the array is (the number of data points).

Numpy (of course) has built in functions to calculate mean with np.mean or np.average.

```
In [26]: x = np.arange(0, 7) #[0,1,2,3,4,5,6]

         mean = np.sum(x)/len(x)
         print(mean)

         mean2 = np.mean(x)
         mean3 = np.average(x)

         print(mean2, mean3)
```

```
3.0
3.0 3.0
```

These should all give the same output: 3. Make sure you also understand how to do that by hand and compare with equation 4-1 in the textbook.

The standard deviation is calculated as shown below. Write this out or compare it with equation 4-2 in the textbook to make sure you understand it.

```
In [27]: std_dev = np.sqrt(np.sum((x - mean)**2)/(len(x-1)))
         print(std_dev)
```

```
2.0
```

Problem 4: Now that you know how to define the standard deviation. Write a function to calculate the standard deviation from only one array input, x. Compare the printout with the numpy function to calculate standard deviation, np.std(). You can use any function you want in your function except np.std(). To clarify, I want to write your own function here using def and the same method outlined above with add() and Gaussian(). It should take only one input. Hint: it should start def StandardDeviation(x):

```
In [28]: #Your code here.

         def StandardDeviation(x):
             mean=np.mean(x)
             std_dev = np.sqrt(np.sum((x - mean)**2)/(len(x-1)))
             return std_dev

         print("My function = ", StandardDeviation(x))
         print("Numpy says = ", np.std(x))
```

```
My function =  2.0
Numpy says =  2.0
```

Ok, now that you have written a function to calculate standard deviation. Let's use it on some mock analytical data.

I've added some code to generate some random electrochemical cell potential data, E. The function np.random.normal lets us generate random data with a mean and standard deviation as defined in the code.

The function plt.hlines() allows us to put a horizontal line on the plot at a specific y value (mean) ranging from a minmum x value (0) to a maximum x value (number). The rest of the code are some parameters for color and line style. These can also be used with the normal plot() command.

Problem 5: This code shows the theoretical mean and standard deviation. Use any of the code above to calculate 1) the experimental mean and 2) the experimental standard deviation for the array E in the code below. Print them out and add the experiemental values as black lines to the plot alongside the red theoretical lines. Challenge: change the number of data points to either much smaller or much larger. How do the experimental and theoretical values compare?

In [33]:
```
mean = 0.777
std_dev = 0.05
number = 100
E = np.random.normal(mean, std_dev, number)

experimental_mean = np.mean(E)
experimental_stddev = np.std(E)

print("Theoretical Mean:", mean)
print("Experimental Mean:", experimental_mean)
print("Theoretical Std Dev:", std_dev)
print("Experimental Std Dev:", experimental_stddev)

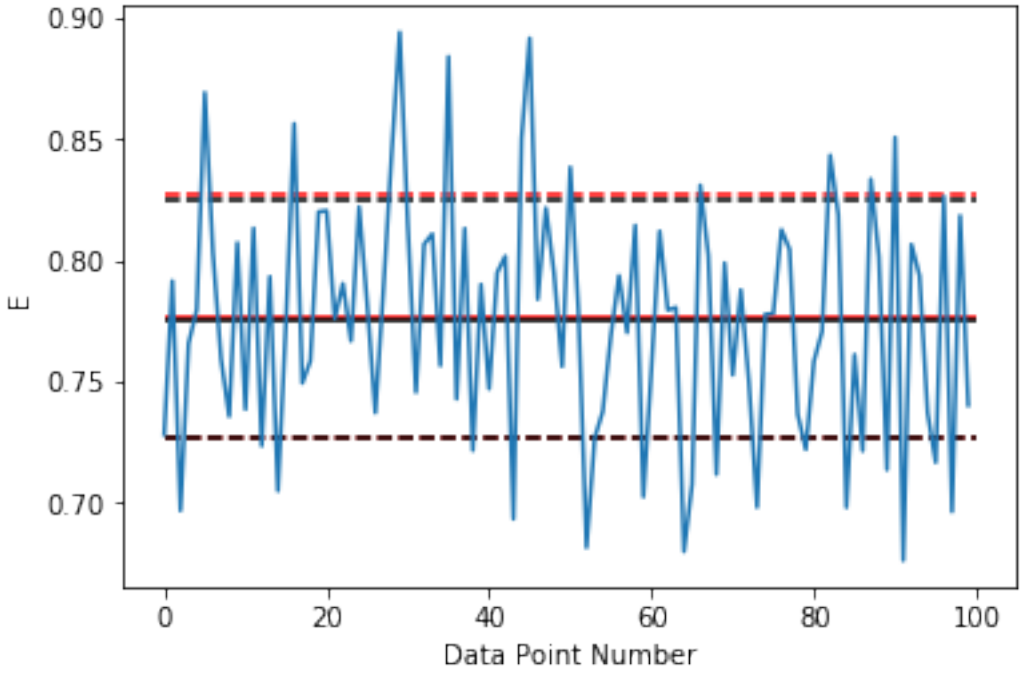
plt.plot(E)
plt.hlines(mean, 0, number, color="red")
plt.hlines(mean + std_dev, 0, number, color="red", linestyle="--")
plt.hlines(mean - std_dev, 0, number, color="red", linestyle="--")

plt.hlines(experimental_mean, 0, number, color="black")
plt.hlines(experimental_mean + experimental_stddev, 0, number, color="black", linestyle
plt.hlines(experimental_mean - experimental_stddev, 0, number, color="black", linestyle

#Your code here...

plt.ylabel("E")
plt.xlabel("Data Point Number")
plt.show()
```

```
Theoretical Mean: 0.777
Experimental Mean: 0.775861679054
Theoretical Std Dev: 0.05
Experimental Std Dev: 0.0488763251534
```



In [35]:
```
mean = 0.777
std_dev = 0.05
number = 1000
E = np.random.normal(mean, std_dev, number)

experimental_mean = np.mean(E)
experimental_stddev = np.std(E)

print("Theoretical Mean:", mean)
print("Experimental Mean:", experimental_mean)
print("Theoretical Std Dev:", std_dev)
print("Experimental Std Dev:", experimental_stddev)
print("These should be closer than they are with fewer data points. More data points me

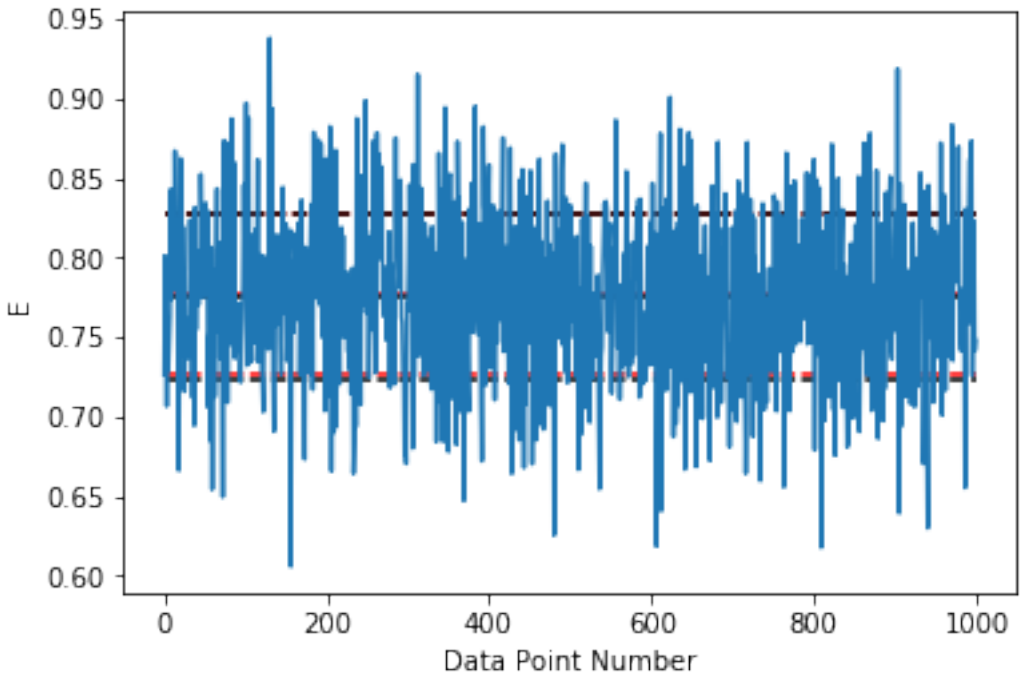
plt.plot(E)
plt.hlines(mean, 0, number, color="red")
plt.hlines(mean + std_dev, 0, number, color="red", linestyle="--")
plt.hlines(mean - std_dev, 0, number, color="red", linestyle="--")

plt.hlines(experimental_mean, 0, number, color="black")
plt.hlines(experimental_mean + experimental_stddev, 0, number, color="black", linestyle
plt.hlines(experimental_mean - experimental_stddev, 0, number, color="black", linestyle

#Your code here...
```

```
        plt.ylabel("E")
        plt.xlabel("Data Point Number")
        plt.show()
```

```
Theoretical Mean: 0.777
Experimental Mean: 0.775034736029
Theoretical Std Dev: 0.05
Experimental Std Dev: 0.0516513587253
These should be closer than they are with fewer data points. More data points mean better estima
```

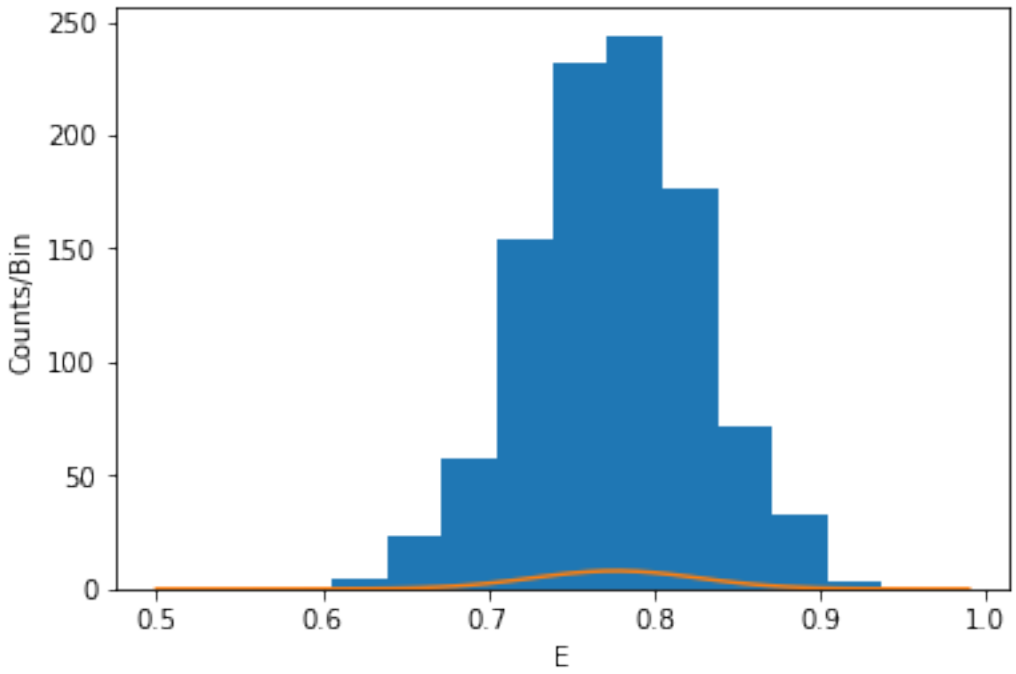


This gives you an example of what might happen in a real electrochemical device. It is constantly taking measurements. The analysis software will synthesize a range of data to calculate the appropriate values. This also shows you why Python is so powerful. You can quickly process a lot of data, do a range of manipulations, and generate really nice plots afterwards.

The code below shows a histogram of the a data points compared with the true underlying statistical Gaussian distribution. Note, it hasn't been normalized so that the areas match.

```
In [36]: x = np.arange(0.5, 1, 0.01)
         y = Gaussian(x, mean, std_dev)

         plt.hist(E)
         plt.plot(x, y)
         plt.xlabel("E")
         plt.ylabel("Counts/Bin")
         plt.show()
```



Part 2: Linear regression.

Another common thing analytical chemists need to do is generate a calibration curve, such as for a pH meter.

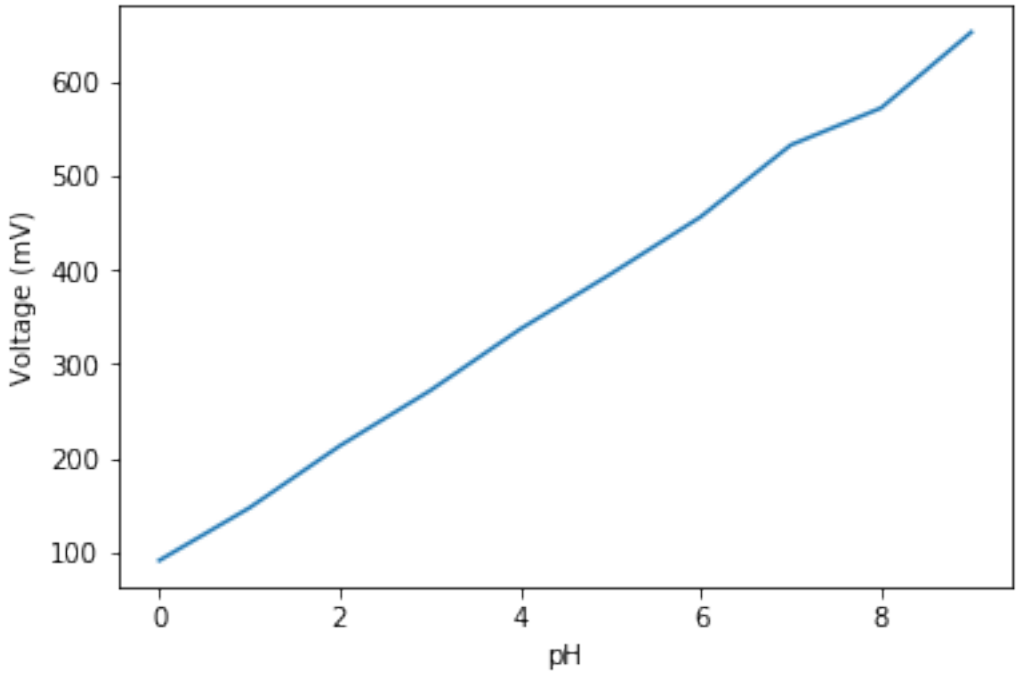
We will start by simulating a linear response of voltage (in mV) to a known pH input. We will add in some noise using the same function above.

We are using the xlabel() and ylabel() functions to write the axes labels.

```
In [37]: slope = 59.16
         intercept = 100

         x = np.arange(0,10)
         y = slope * x + intercept
         noise = np.random.normal(0, 10, len(x))
         y = y + noise

         plt.plot(x, y)
         plt.xlabel("pH")
         plt.ylabel("Voltage (mV)")
         plt.show()
```



Now, imagine we didn't know the slope and the intercept. We need to calculate this from the x and y data.

To do this, we will use numpy's polyfit command. It can fit any order polynomial, but here we will use an order of 1 to get a linear fit. It returns an array (fit) of the polynomial coefficients in decending order. Thus, the slope is the first value in the array, and the intercept is the second value. These should be close to the values above. If you were to remove the noise (for example by commenting out the line that added it in or by reducing the standard deviation of the noise to 0), you should have values that perfectly match the inputs.

To access individual values from an array, we use []. In computer programming, the first value is actually position 0. So, fit[0] gives you the slope or first element of array. Using fit[1] will extract the second value of the array.

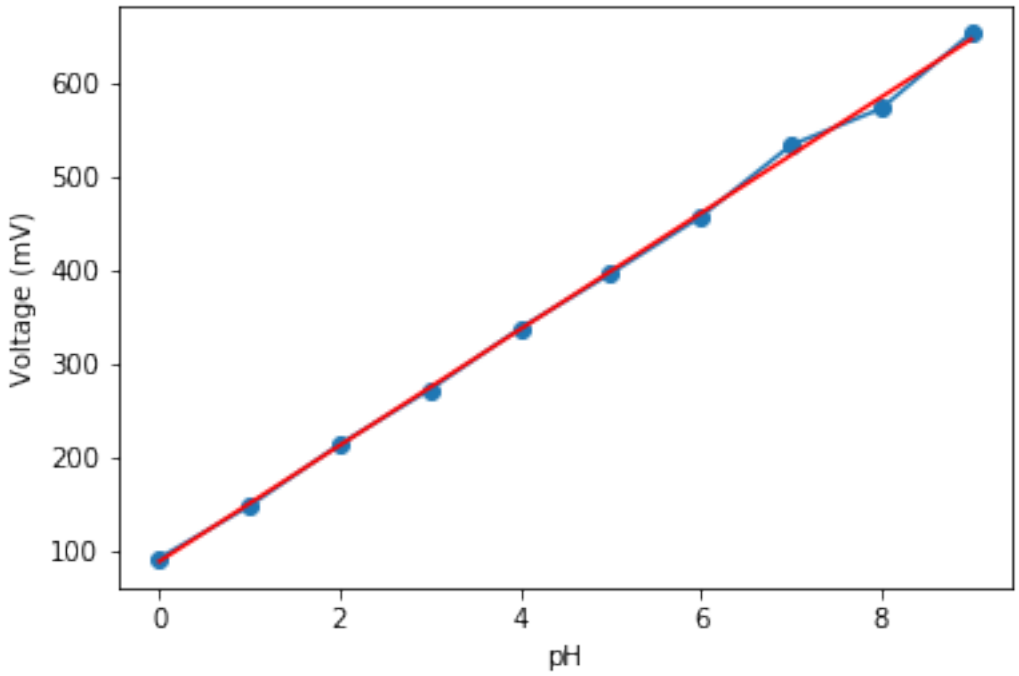
```
In [38]: fit = np.polyfit(x,y, 1)
         print(fit)

         fit_intercept = fit[1]
         fit_slope = fit[0]

         fit_ydata = fit_slope * x + fit_intercept

         plt.plot(x, y, marker="o")
         plt.plot(x, fit_ydata, color="red")
         plt.xlabel("pH")
         plt.ylabel("Voltage (mV)")
         plt.show()
```

```
[ 62.04069754  87.8381738 ]
```



Problem 6: Now that you know how to fit a calibration curve, write a function to calculate the pH (x value) from a measured voltage (y value) based on the calibration parameters you determined. Solve for a measured voltage of 325 mV. Challenge: plot the measured value on the same plot above. Challenge 2: Calculate the uncertainty from equation 4-27 in the textbook.

```
In [41]: # Your code here...
         fit = np.polyfit(x,y, 1)
         fit_intercept = fit[1]
         fit_slope = fit[0]

         def calc_fit(y, slope, intercept):
             return (y - intercept)/slope

         pH = calc_fit(325, fit_slope, fit_intercept)
         print("pH = ", pH)

         plt.plot(x, y, marker="o")
         plt.plot(x, fit_ydata, color="red")
         plt.plot(pH, 325, marker="o", color="green")
         plt.xlabel("pH")
         plt.ylabel("Voltage (mV)")
         plt.show()
```

```
pH =  3.82268149124
```

